

```
#include <stdio.h>           // Standard I/O functions (printf, fprintf, etc.)
#include <string.h>          // String handling (strlen, strcpy, memcpy, strcmp)
#include <stdlib.h>          // General utilities (exit, malloc, etc.)
#include <signal.h>          // Signal handling (signal, SIGINT)
#include <unistd.h>          // POSIX APIs (sleep, close, read, write)
#include <net/if.h>          // Network interface structures (ifreq)
#include <sys/ioctl.h>        // IO control (ioctl, SIOCGIFINDEX)
#include <sys/socket.h>       // Socket APIs (socket, bind, select)
#include <linux/can.h>        // CAN bus frame definitions (struct can_frame)
#include <linux/can/raw.h>    // CAN RAW socket options
#include "MQTTClient.h"      // Paho MQTT C client API

#define BROKER      "tcp://test.mosquitto.org:1883" // MQTT broker URL
#define CLIENTID    "tk6wXkStDk"                 // MQTT client identifier
#define QOS         1                             // MQTT QoS level to use
#define TMO_MS      10000L                         // Publish/complete timeout (ms)

#define TOP_TEMP     "pi3_ctrl/temp"               // Topic for temperature telemetry
#define TOP_LED      "pi3_ctrl/led"                // Topic for LED state (reported)
#define TOP_RELAY    "pi3_ctrl/relay"              // Topic for relay state (reported)
#define TOP_LED_SW   "pi3_ctrl/led_switch"         // Topic for LED switch commands
(incoming)
#define TOP_RELAY_SW "pi3_ctrl/relay_switch"       // Topic for relay switch commands
(incoming)

#define CAN_DEV      "can0"                        // Linux CAN interface name
#define ID_STAT      0x210                         // CAN ID for status frames (from ESP32 → host)
#define ID_CMD       0x211                         // CAN ID for command frames (host → ESP32)

volatile int running = 1;                          // Main loop flag; cleared by SIGINT handler
int canfd = -1; /* CAN socket */ // File descriptor for CAN RAW socket
MQTTClient cli; /* MQTT client */ // MQTT client handle

static int mqtt_pub(const char *topic, const char *payload, int retain) // Helper: publish a message
{
    MQTTClient_message msg = MQTTClient_message_initializer; // Zero/Default init
    MQTT message
    msg.payload = (void *)payload; // Point payload to provided buffer
    msg.payloadlen = (int)strlen(payload); // Set payload length
    msg.qos = QOS; // Use configured QoS
    msg.retained = retain; // Set retained flag if requested
    MQTTClient_deliveryToken tok; // Delivery token for async publish
    int rc = MQTTClient_publishMessage(cli, topic, &msg, &tok); // Publish async
    if (rc == MQTTCLIENT_SUCCESS) // If queued successfully,
        rc = MQTTClient_waitForCompletion(cli, tok, TMO_MS); // wait up to TMO_MS for
completion
    return rc; // Return Paho result code
}
```

```

static void sigint(int s){ running = 0; } // SIGINT (Ctrl-C) handler: stop loop

int main(void) // Program entry point
{
    /* ----- CAN ----- */
    signal(SIGINT, sigint); // Register SIGINT handler
    canfd = socket(PF_CAN, SOCK_RAW, CAN_RAW); // Open a RAW CAN
socket
    struct ifreq ifr; strcpy(ifr.ifr_name, CAN_DEV); // Specify interface name (can0)
    ioctl(canfd, SIOCGIFINDEX, &ifr); // Get interface index for can0
    struct sockaddr_can caddr={ .can_family=AF_CAN,.can_ifindex=ifr.ifr_ifindex }; // Build CAN
addr
    bind(canfd,(struct sockaddr*)&caddr,sizeof(caddr)); // Bind socket to CAN interface

    /* ----- MQTT ----- */
    MQTTClient_connectOptions copts = MQTTClient_connectOptions_initializer; // Default
connection opts
    MQTTClient_create(&cli, BROKER, CLIENTID, MQTTCLIENT_PERSISTENCE_NONE,
NULL); // Create MQTT client
    copts.keepAliveInterval = 60; copts.cleansession = 1; // Set keepalive and clean session
mqtt_reconnect: // Label: retry connect on failure
    if (MQTTClient_connect(cli,&copts)!=MQTTCLIENT_SUCCESS){ // Attempt to
connect to broker
        fprintf(stderr,"MQTT connect fail – retry\n"); sleep(5); goto mqtt_reconnect; // Backoff &
retry
    }
    MQTTClient_subscribe(cli, TOP_LED_SW , QOS); // Subscribe to LED switch
commands
    MQTTClient_subscribe(cli, TOP_RELAY_SW, QOS); // Subscribe to relay switch
commands
    puts("Bridge running"); // Inform that the bridge is up

    int ledState=-1, relayState=-1; // Cached states (unknown initially)
    char buf[16]; // Small buffer for formatted temp
    struct can_frame fr; // Buffer for incoming CAN frames

    while(running){ // Main loop until SIGINT
        /* ----- poll MQTT (non-blocking) ----- */
        char *tpc=NULL; int tlen; MQTTClient_message *msg=NULL; // Placeholders for
received msg
        int rc=MQTTClient_receive(cli,&tpc,&tlen,&msg,0); // 0-ms timeout */ // Poll without
blocking
        if(rc==MQTTCLIENT_SUCCESS && msg){ // got a switch cmd */ // If a message
arrived
            char topic[64]={0}; if(tlen>63)tlen=63; memcpy(topic,tpc,tlen); // Copy topic safely
(truncate)
            char val=((char*)msg->payload)[0]; // First payload char ('0' or '1')
            struct can_frame cmd={ .can_id=ID_CMD,.can_dlc=2,.data={0xFF,0xFF}}; // Prepare
command frame
            if(!strcmp(topic,TOP_LED_SW)) cmd.data[0]=val-'0'; // Set LED field if topic
matches

```

```

        if(!strcmp(topic,TOP_RELAY_SW)) cmd.data[1]=val-'0';        // Set relay field if topic
matches
        write(canfd,&cmd,sizeof(cmd));        /* forward to ESP32 */ // Send command on CAN
        printf("SWITCH %s = %c → CAN 0x211 [%02X %02X]\n",        // Log what was sent
            topic,val,cmd.data[0],cmd.data[1]);
        MQTTClient_freeMessage(&msg); MQTTClient_free(tpc);        // Free Paho-allocated
objects
    }

    /* ----- read CAN (non-blocking) ----- */
    struct timeval tv={.tv_sec=0,.tv_usec=10000}; /* 10 ms */        // Select timeout (10 ms)
    fd_set fds; FD_ZERO(&fds); FD_SET(canfd,&fds);        // Prepare fd set for select()
    if(select(canfd+1,&fds,NULL,NULL,&tv)>0 &&        // If CAN fd is ready to read
        read(canfd,&fr,sizeof(fr))==sizeof(fr) &&        // And a full frame was read
        fr.can_id==ID_STAT && fr.can_dlc==4)        // And it is a status frame
    {
        int16_t ti=(fr.data[0]<<8)|fr.data[1];        // Temperature in hundredths °C
        int nLed=fr.data[2], nRel=fr.data[3];        // LED/Relay states from frame

        snprintf(buf,sizeof(buf),"%0.2f",ti/100.0f);        // Format temperature as string
        mqtt_pub(TOP_TEMP ,buf ,0);        // Publish temperature (not retained)

        if(nLed!=ledState){ mqtt_pub(TOP_LED ,nLed?"1":"0",1); ledState=nLed; }    // Publish
LED change
        if(nRel!=relayState){ mqtt_pub(TOP_RELAY,nRel?"1":"0",1); relayState=nRel;} // Publish
relay change
    }
}
MQTTClient_disconnect(cli,10000); MQTTClient_destroy(&cli); close(canfd); // Cleanup
MQTT and CAN
return 0;        // Exit success
}

```

esp32_can_slave_ctrl

```

#include <Wire.h>        // I2C/TWI library for communicating with the LM75 temperature
sensor
#include <SPI.h>        // SPI library used by the MCP2515 CAN controller
#include <mcp2515.h>        // MCP2515 CAN controller driver

#define LM75_ADDR 0x48        // 7-bit I2C address of the LM75 temperature sensor
#define LED_PIN 32        // GPIO pin driving the LED
#define RELAY_PIN 33        // GPIO pin driving the relay (active-LOW hardware)

#define CAN_STAT_ID 0x210        // CAN identifier for status frames sent out
#define CAN_CMD_ID 0x211        // CAN identifier for command frames received
#define PERIOD_MS 5000        // Periodic status transmit interval (milliseconds)

MCP2515 can(5);        // MCP2515 object on CS pin 5

```

```

struct can_frame tx, rx;      // CAN frame buffers: tx for transmit, rx for receive
bool ledOn = false, relayOn = false; // Cached output states (LED and relay)

float readLM75() {           // Read temperature in °C from the LM75 over I²C
    Wire.beginTransmission(LM75_ADDR); // Start an I²C write to the LM75
    Wire.write(0x00); Wire.endTransmission(false); // Select temperature register (0x00), repeated
start
    Wire.requestFrom(LM75_ADDR, 2); // Request 2 bytes of temperature data
    if (Wire.available() < 2) return NAN; // If not enough bytes, return NaN to signal error
    uint16_t raw = (Wire.read() << 8) | Wire.read(); // Combine MSB and LSB into 11-bit
left-justified value
    return (int16_t)(raw >> 5) * 0.125f; // Convert to °C (LM75: 0.125°C/LSB, 11-bit signed)
}

void applyIO() {             // Drive the physical outputs according to cached states
    digitalWrite(LED_PIN, ledOn ? HIGH : LOW); // LED is active-HIGH
    digitalWrite(RELAY_PIN, relayOn ? LOW : HIGH); // Relay is active-LOW (invert logic)
}

void sendStatus(float t) {    // Pack and transmit a status frame with temperature and IO states
    int16_t ti = (int16_t)(t * 100.0f); // Temperature scaled to centi-degrees (×100)
    tx.can_id = CAN_STAT_ID; // Set status frame CAN ID
    tx.can_dlc = 4; // Payload length = 4 bytes
    tx.data[0] = (ti >> 8) & 0xFF; // Temperature high byte
    tx.data[1] = ti & 0xFF; // Temperature low byte
    tx.data[2] = ledOn ? 1 : 0; // LED state (1=on, 0=off)
    tx.data[3] = relayOn ? 1 : 0; // Relay state (1=on, 0=off)
    can.sendMessage(&tx); // Transmit the status frame over CAN
}

void setup() {               // Arduino/ESP32 setup runs once at boot
    Serial.begin(115200); // Initialize serial for debugging
    pinMode(LED_PIN, OUTPUT); pinMode(RELAY_PIN, OUTPUT); // Configure IO pins as
outputs
    applyIO(); // Apply initial output states
    Wire.begin(); // Initialize I²C bus
    SPI.begin(); // Initialize SPI bus for MCP2515
    can.reset(); can.setBaudrate(CAN_500KBPS, MCP_8MHZ); can.setNormalMode(); // Reset, set 500
kbps @ 8 MHz, go to normal mode
    tx.can_id = CAN_STAT_ID; tx.can_dlc = 4; // Preconfigure TX frame ID and DLC for
status messages
}

void loop() {                // Main loop runs repeatedly
    /* handle incoming commands */
    while (can.readMessage(&rx) == MCP2515::ERROR_OK) { // Drain all received CAN frames
        if (rx.can_id == CAN_CMD_ID && rx.can_dlc == 2) { // Process only 2-byte command frames
with expected ID
            bool changed = false; // Track whether any output actually changed
            if (rx.data[0] != 0xFF) { ledOn = rx.data[0]; changed = true; } // If LED byte set (not 0xFF),
update LED state
            if (rx.data[1] != 0xFF) { relayOn = rx.data[1]; changed = true; } // If relay byte set (not 0xFF),
update relay state
            if (changed) { applyIO(); sendStatus(readLM75()); } // Apply changes and report
updated status
        }
    }
}

```

```

}
/* periodic status */
static uint32_t last = 0;           // Timestamp of last periodic transmission
if (millis() - last >= PERIOD_MS) { // If the period has elapsed
    last = millis(); sendStatus(readLM75()); // Refresh timestamp and send current status
}
}

```

esp32_can_master_ctrl

```

#include <WiFi.h>           // ESP32 Wi-Fi stack (station mode, connection status, etc.)
#include <PubSubClient.h>   // Lightweight MQTT client for Arduino/ESP32
#include <SPI.h>            // SPI bus support used by the MCP2515 CAN controller
#include <mcp2515.h>        // Driver for MCP2515 stand-alone CAN controller over SPI

/* ----- user config ----- */
#define WIFI_SSID "huawei_9761" // Wi-Fi network SSID to connect to
#define WIFI_PASS "2651031148" // Wi-Fi network password

#define MQTT_HOST "test.mosquitto.org" // MQTT broker hostname (public Mosquitto test broker)
#define MQTT_PORT 1883                // MQTT broker TCP port (unencrypted)

/* ----- topics ----- */
const char* TOP_TEMP = "pi3_ctrl/temp"; // Topic for temperature telemetry (float as string)
const char* TOP_LED = "pi3_ctrl/led";    // Topic publishing LED state ("0"/"1", retained)
const char* TOP_RELAY = "pi3_ctrl/relay"; // Topic publishing relay state ("0"/"1", retained)
const char* TOP_LED_SW = "pi3_ctrl/led_switch"; // Topic to receive LED switch commands ("0"/"1")
const char* TOP_RELAY_SW = "pi3_ctrl/relay_switch"; // Topic to receive relay switch commands ("0"/"1")

/* ----- CAN IDs ----- */
#define CAN_STAT_ID 0x210 // CAN identifier for status frames coming from the sensor node
#define CAN_CMD_ID 0x211 // CAN identifier for command frames sent to the sensor node

/* ----- MCP2515 ----- */
MCP2515 mcp2515(5); // Create MCP2515 object; chip-select (CS) on GPIO 5
/* INT on GPIO16 – not used for ISR here, but wiring it allows future interrupts */ // Note about optional INT pin wiring

/* ----- Wi-Fi / MQTT ----- */
WiFiClient wifi; // TCP client used by PubSubClient for MQTT transport
PubSubClient mqtt(wifi); // MQTT client bound to the Wi-Fi TCP client

/* publish helper (QoS 0, retained = true for led/relay, false for temp) */
void pub(const char* topic, const char* payload, bool retain=false) // Helper to publish a message
{
    mqtt.publish(topic, payload, retain); // Publish with QoS 0 and given retain flag
}

```

```

} // end pub

/* forward dashboard switch → CAN command */
void mqttCallback(char* topic, byte* payload, unsigned int len) // Called by PubSubClient on
inbound msgs
{
  if (len==0) return;          // Ignore empty payloads
  char val = payload[0];       // Read first character ('0' or '1')
  byte data[2] = { 0xFF, 0xFF }; // Default: no-change markers for LED and relay

  if (val!='0' && val!='1') return; // Validate payload; accept only '0' or '1'
  if (!strcmp(topic, TOP_LED_SW)) data[0] = (val=='1'); // If LED topic, set LED byte (0/1)
  if (!strcmp(topic, TOP_RELAY_SW)) data[1] = (val=='1'); // If relay topic, set relay byte (0/1)

  struct can_frame cmd;        // Build a CAN command frame
  cmd.can_id = CAN_CMD_ID;     // Use command CAN ID
  cmd.can_dlc = 2;             // Two data bytes: LED, RELAY
  cmd.data[0] = data[0];        // First byte: LED command or 0xFF (no change)
  cmd.data[1] = data[1];        // Second byte: RELAY command or 0xFF (no change)
  mcp2515.sendMessage(&cmd);    // Transmit the command over CAN
} // end mqttCallback

/* reconnect Wi-Fi and MQTT if needed */
void ensureMQTT()              // Keep connectivity alive; reconnect as required
{
  if (WiFi.status()!=WL_CONNECTED) // If Wi-Fi is disconnected,
  {
    WiFi.begin(WIFI_SSID, WIFI_PASS); // start connection to the configured AP
    while (WiFi.status()!=WL_CONNECTED) delay(500); // wait (polling) until connected
  } // end Wi-Fi reconnect

  if (!mqtt.connected())        // If MQTT is not connected,
  {
    while (!mqtt.connected())    // loop until we establish a session
      mqtt.connect("esp32_can_master"); // connect with a simple client ID (no auth/TLS)
    mqtt.subscribe(TOP_LED_SW);   // Subscribe to LED switch command topic
    mqtt.subscribe(TOP_RELAY_SW); // Subscribe to relay switch command topic
  } // end MQTT reconnect
} // end ensureMQTT

void setup()                   // Arduino setup: runs once at boot
{
  Serial.begin(115200);        // Initialize serial console for logs

  /* Wi-Fi first */
  WiFi.begin(WIFI_SSID, WIFI_PASS); // Start connecting to the configured Wi-Fi network
  while (WiFi.status()!=WL_CONNECTED) delay(500); // Block until connected

  /* MQTT */
  mqtt.setServer(MQTT_HOST, MQTT_PORT); // Configure MQTT broker address/port
  mqtt.setCallback(mqttCallback);       // Register callback for incoming MQTT messages

```

```

/* SPI + MCP2515 */
SPI.begin(23, 19, 18, 5);    // Initialize SPI with custom pins (SCK, MISO, MOSI, SS)
mcp2515.reset();            // Reset MCP2515 controller
mcp2515.setBtrrate(CAN_500KBPS, MCP_8MHZ); // Configure CAN bit rate: 500 kbps with 8
MHz crystal
mcp2515.setNormalMode();    // Enter normal operating mode (RX/TX enabled)
} // end setup

void loop()                // Main loop: runs repeatedly
{
  ensureMQTT();            // Make sure Wi-Fi/MQTT links are up
  mqtt.loop();             // Let PubSubClient process incoming/outgoing MQTT packets

  /* read CAN frames (polling) */
  struct can_frame rx;     // Buffer for a received CAN frame
  if (mcp2515.readMessage(&rx) == MCP2515::ERROR_OK && // If a frame is available,
      rx.can_id == CAN_STAT_ID && rx.can_dlc == 4) // and it's a 4-byte status frame from node
  {
    int16_t ti = (rx.data[0] << 8) | rx.data[1]; // Reconstruct temperature in centi-degrees
    char buf[16]; // Temporary string buffer
    snprintf(buf, sizeof(buf), "%.2f", ti/100.0); // Format temperature as "xx.yy"
    pub(TOP_TEMP, buf, false); // not retained // Publish current temperature
    (non-retained)

    pub(TOP_LED, rx.data[2]?"1":"0", true); // retained so widgets populate // Publish LED state
    (retained)
    pub(TOP_RELAY, rx.data[3]?"1":"0", true); // Publish relay state (retained)
  } // end if frame available
} // end loop

```

raspberrypi3_can_slave_ctrl

```

#include <stdio.h> // Standard I/O (printf, fprintf)
#include <stdlib.h> // General utilities (exit, atoi, etc.)
#include <string.h> // String handling (strcpy, memset, etc.)
#include <signal.h> // Signal handling (signal, SIGINT)
#include <unistd.h> // POSIX APIs (read, write, close, usleep)
#include <wiringPi.h> // GPIO access via wiringPi (pinMode, digitalWrite, millis)
#include <wiringPiI2C.h> // I2C helper functions for wiringPi (wiringPiI2CSetup, ReadReg16)
#include <net/if.h> // Network interface structures (ifreq)
#include <sys/ioctl.h> // IO control calls (ioctl, SIOCGIFINDEX)
#include <sys/socket.h> // Socket API (socket, bind, select)
#include <linux/can.h> // Linux SocketCAN frame definitions (struct can_frame)
#include <linux/can/raw.h> // RAW CAN socket options and constants

#define LM75_ADDR 0x48 // 7-bit I2C address of LM75 temperature sensor
#define LED_PIN 0 // wiringPi pin 0 (BCM GPIO17) used for LED output
#define REL_PIN 2 // wiringPi pin 2 (BCM GPIO27) used for RELAY output
(active-LOW)

#define ID_STAT 0x210 // CAN identifier for status frames we transmit

```

```

#define ID_CMD 0x211          // CAN identifier for command frames we receive
#define PERIOD_MS 5000        // Periodic status interval in milliseconds

volatile int running = 1;     // Global flag to keep main loop running; cleared on SIGINT
void sigint(int s){ running = 0; } // SIGINT (Ctrl-C) handler: request graceful shutdown

static float read_lm75(int fd) // Read temperature (°C) from LM75 using its I2C file descriptor
{
    int raw = wiringPiI2CReadReg16(fd, 0x00);    // Read 16-bit temperature register (address 0x00)
    raw = (raw >> 8) | ((raw & 0xFF) << 8);      // LM75 returns big-endian; swap to host endianness
    return (int16_t)(raw >> 5) * 0.125f;         // Convert 11-bit signed value to °C (0.125 °C/LSB)
}

int main(void)                // Program entry point
{
    signal(SIGINT, sigint); // Install SIGINT handler so Ctrl-C stops the loop
    wiringPiSetup();         // Initialize wiringPi library and pin numbering

    pinMode(LED_PIN, OUTPUT); // Configure LED pin as output
    pinMode(REL_PIN, OUTPUT); // Configure relay pin as output (hardware is active-LOW)

    int lm75 = wiringPiI2CSetup(LM75_ADDR);    // Open I2C device for LM75 at address 0x48

    /* open CAN */
    int s = socket(PF_CAN, SOCK_RAW, CAN_RAW); // Create a RAW CAN socket
    struct ifreq ifr; strcpy(ifr.ifr_name, "can0"); // Specify the CAN interface name ("can0")
    ioctl(s, SIOCGIFINDEX, &ifr);                // Look up numeric interface index for "can0"
    struct sockaddr_can addr = { .can_family = AF_CAN,
                                  .can_ifindex = ifr.ifr_ifindex }; // Build CAN socket address (family + ifindex)
    bind(s, (struct sockaddr*)&addr, sizeof(addr)); // Bind the socket to the CAN interface

    struct can_frame tx = { .can_id = ID_STAT, .can_dlc = 4 }; // Preconfigure TX frame for status (4 data bytes)
    struct can_frame rx; // Buffer for incoming CAN frames
    uint32_t last = 0; // Timestamp of last periodic publish (ms)

    while (running) // Main loop until SIGINT sets running=0
    {
        /* ----- handle incoming command ----- */
        struct timeval tv = { .tv_sec = 0, .tv_usec = 1000 }; // select() timeout: 1 ms (non-blocking-ish)
        fd_set fds; FD_ZERO(&fds); FD_SET(s, &fds); // Prepare fd set with CAN socket
        if (select(s+1, &fds, NULL, NULL, &tv) > 0 && // If socket readable within timeout
            read(s, &rx, sizeof(rx)) == sizeof(rx) && // And a full CAN frame was read
            rx.can_id == ID_CMD && rx.can_dlc == 2) // And it matches our command ID and length
        {
            if (rx.data[0] != 0xFF) digitalWrite(LED_PIN, rx.data[0]); // Update LED if not "no-change" (0xFF)

```



```

        if (rx.data[1] != 0xFF) digitalWrite(REL_PIN , rx.data[1] ? LOW : HIGH); // Update relay;
active-LOW hardware
    /* echo new status immediately */
    float t = read_lm75(lm75);           // Read current temperature from LM75
    int16_t ti = (int16_t)(t*100);       // Scale to centi-degrees for transport
    tx.data[0]=ti>>8; tx.data[1]=ti&0xFF; // Pack temperature MSB/LSB
    tx.data[2]=digitalRead(LED_PIN);     // Pack current LED state (0/1)
    tx.data[3]=!digitalRead(REL_PIN);    // Pack relay state (invert because
active-LOW)
    write(s,&tx,sizeof(tx));              // Transmit immediate status frame on CAN
}

/* ----- periodic status ----- */
if (millis() - last >= PERIOD_MS){      // If it's time for the periodic update
    last = millis();                    // Update last-sent timestamp
    float t = read_lm75(lm75);          // Read latest temperature
    int16_t ti = (int16_t)(t*100);      // Convert °C to centi-degrees
    tx.data[0]=ti>>8; tx.data[1]=ti&0xFF; // Fill temperature bytes
    tx.data[2]=digitalRead(LED_PIN);    // Fill LED state
    tx.data[3]=!digitalRead(REL_PIN);   // Fill relay state (active-LOW → logical
invert)
    write(s,&tx,sizeof(tx));             // Send status frame over CAN
}
}
close(s);                               // Close CAN socket on exit
return 0;                               // Return success
}

```

